Journal Assignment – 4

Q:- 1. Setup - Database Connection and Table Creation: 1. Connect to Database, 2. Create tasks Table.

- **Input:-**
- **step1_setup_todo_db.py:-**

```
import sqlite3

def setup_todo_database():
    # 1. Connect to Database (creates 'todo_app.db' if it does not exist)
    conn = sqlite3.connect('todo_app.db')
    cursor = conn.cursor()
    print("Connected to database 'todo_app.db' successfully.")

    # 2. Create 'tasks' Table
    cursor.execute('''
    CREATE TABLE IF NOT EXISTS tasks (
        task_id INTEGER PRIMARY KEY AUTOINCREMENT,
        description TEXT NOT NULL,
        due_date TEXT NOT NULL,
        is_completed INTEGER DEFAULT 0,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    );
    ''')
    conn.commit()
    print("Table 'tasks' created successfully with fields: task_id, description, due_date,
is_completed, created_at.")
    conn.close()
    print("Connection closed.")

if __name__ == '__main__':
    setup_todo_database()
```

- **Output:-**

```
Connected to database 'todo_app.db' successfully.
Table 'tasks' created successfully with fields: task_id, description, due_date, is_completed,
created_at.
Connection closed.
```

Q:- 2. Core To-Do List Functions: 1. add_task(description, due_date), 2. view_tasks(show_completed=False), 3. mark_task_completed(task_id), 4. delete_task(task_id).

- **Input:-**
- **step2_todo_functions.py:-**

```
import sqlite3

DB_NAME = 'todo_app.db'

def get_connection():
    return sqlite3.connect(DB_NAME)

# 1. add_task(description, due_date) function
def add_task(description, due_date):
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute('''
    INSERT INTO tasks (description, due_date, is_completed)
    VALUES (?, ?, 0);
    ''', (description, due_date))
    conn.commit()
    task_id = cursor.lastrowid
    conn.close()
    print(f"Task #{task_id} ('{description}') added successfully.")
    return task_id

# 2. view_tasks(show_completed=False) function
def view_tasks(show_completed=False):
    conn = get_connection()
    cursor = conn.cursor()
    if show_completed:
        cursor.execute("SELECT task_id, description, due_date, is_completed FROM tasks
ORDER BY task_id;")
    else:
        cursor.execute("SELECT task_id, description, due_date, is_completed FROM tasks
WHERE is_completed = 0 ORDER BY task_id;")

    rows = cursor.fetchall()
    conn.close()

    print("-" * 65)
    print(f"{'ID':<5} {'Description':<32} {'Due Date':<14} {'Status'}")
```

```
    print("-" * 65)
    if not rows:
        print("No tasks found.")
    else:
        for row in rows:
            status = "Completed" if row[3] == 1 else "Pending"
            print(f"{row[0]:<5} {row[1]:<32} {row[2]:<14} {status}")
    print("-" * 65)

# 3. mark_task_completed(task_id) function
def mark_task_completed(task_id):
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("UPDATE tasks SET is_completed = 1 WHERE task_id = ?;", (task_id,))
    conn.commit()
    rows_affected = cursor.rowcount
    conn.close()
    if rows_affected > 0:
        print(f"Task #{task_id} marked as completed.")
    else:
        print(f"Task #{task_id} not found.")

# 4. delete_task(task_id) function
def delete_task(task_id):
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("DELETE FROM tasks WHERE task_id = ?;", (task_id,))
    conn.commit()
    rows_affected = cursor.rowcount
    conn.close()
    if rows_affected > 0:
        print(f"Task #{task_id} deleted successfully.")
    else:
        print(f"Task #{task_id} not found.")

# Demonstration
if __name__ == '__main__':
    t1 = add_task("Complete ML Practical Assignment", "2026-10-25")
    t2 = add_task("Prepare Database Presentation", "2026-10-28")
    t3 = add_task("Submit Web Tech Project", "2026-10-30")

    print("\nPending Tasks:")
    view_tasks(show_completed=False)
```

```
print("\nMarking Task #1 as completed:")
mark_task_completed(t1)
print("\nAll Tasks (including completed):")
view_tasks(show_completed=True)
print("\nDeleting Task #3:")
delete_task(t3)

print("\nRemaining Tasks:")
view_tasks(show_completed=True)
```

- **Output:-**

```
Task #1 ('Complete ML Practical Assignment') added successfully.
Task #2 ('Prepare Database Presentation') added successfully.
Task #3 ('Submit Web Tech Project') added successfully.
 Pending Tasks:
------------------------------------------------------------
ID  Description                 Due Date    Status
------------------------------------------------------------
1   Complete ML Practical Assignment 2026-10-25   Pending
2   Prepare Database Presentation  2026-10-28   Pending
3   Submit Web Tech Project       2026-10-30   Pending
------------------------------------------------------------
 Marking Task #1 as completed:
Task #1 marked as completed.
 All Tasks (including completed):
------------------------------------------------------------
ID  Description                 Due Date    Status
------------------------------------------------------------
1   Complete ML Practical Assignment 2026-10-25   Completed
2   Prepare Database Presentation  2026-10-28   Pending
3   Submit Web Tech Project       2026-10-30   Pending
------------------------------------------------------------

Deleting Task #3:
Task #3 deleted successfully.

Remaining Tasks:
------------------------------------------------------------
ID  Description                 Due Date    Status
------------------------------------------------------------
1   Complete ML Practical Assignment 2026-10-25   Completed
2   Prepare Database Presentation  2026-10-28   Pending
------------------------------------------------------------
```

Q:- 3. Simple User Interface (Main Program Loop): 1. Display Menu (6 options), 2. Enter choice, 3. Process Choice, 4. Error Handling for Input.

- **Input:-**
- **step3_todo_cli_app.py:-**

```
import sqlite3
import sys

DB_NAME = 'todo_app.db'

def get_connection():
    return sqlite3.connect(DB_NAME)

def display_menu():
    print("\n" + "=" * 32)
    print("      To-Do List Manager")
    print("=" * 32)
    print("1. Add New Task")
    print("2. View Pending Tasks")
    print("3. View All Tasks (including completed)")
    print("4. Mark Task as Completed")
    print("5. Delete Task")
    print("6. Exit")
    print("=" * 32)

def add_task(description, due_date):
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("INSERT INTO tasks (description, due_date, is_completed) VALUES (?, ?,
0);", (description, due_date))
    conn.commit()
    task_id = cursor.lastrowid
    conn.close()
    print(f"Success: Task #{task_id} added.")

def view_tasks(show_completed=False):
    conn = get_connection()
    cursor = conn.cursor()
    if show_completed:
        cursor.execute("SELECT task_id, description, due_date, is_completed FROM tasks
ORDER BY task_id;")
    else:
```

```
        cursor.execute("SELECT task_id, description, due_date, is_completed FROM tasks
WHERE is_completed = 0 ORDER BY task_id;")
    rows = cursor.fetchall()
    conn.close()

    print("\nID   Status     Due Date      Description")
    print("-" * 55)
    if not rows:
        print("No tasks found.")
    for r in rows:
        st = "[DONE]" if r[3] == 1 else "[TODO]"
        print(f"{r[0]:<4} {st:<10} {r[2]:<13} {r[1]}")
    print("-" * 55)

def mark_task_completed(task_id):
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("UPDATE tasks SET is_completed = 1 WHERE task_id = ?;", (task_id,))
    conn.commit()
    updated = cursor.rowcount
    conn.close()
    if updated > 0:
        print(f"Success: Task #{task_id} marked as completed.")
    else:
        print(f"Error: Task #{task_id} does not exist.")

def delete_task(task_id):
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("DELETE FROM tasks WHERE task_id = ?;", (task_id,))
    conn.commit()
    deleted = cursor.rowcount
    conn.close()
    if deleted > 0:
        print(f"Success: Task #{task_id} deleted successfully.")
    else:
        print(f"Error: Task #{task_id} does not exist.")

def main_loop(simulated_inputs=None):
    # Simulated inputs allow demonstrating menu flow deterministically
    input_index = 0
    def custom_input(prompt):
        nonlocal input_index
        if simulated_inputs is not None and input_index < len(simulated_inputs):
```

```
        val = simulated_inputs[input_index]
        print(f"{prompt}{val}")
        input_index += 1
        return val
    return input(prompt)

while True:
    display_menu()
    try:
        choice = custom_input("Enter your choice: ").strip()

        # Error Handling for Empty Input
        if not choice:
            print("Invalid input. Please enter a number between 1 and 6.")
            continue

        if choice == '1':
            desc = custom_input("Enter task description: ").strip()
            if not desc:
                print("Error: Task description cannot be empty.")
                continue
            due = custom_input("Enter due date (YYYY-MM-DD): ").strip()
            add_task(desc, due)

        elif choice == '2':
            view_tasks(show_completed=False)

        elif choice == '3':
            view_tasks(show_completed=True)

        elif choice == '4':
            task_id_str = custom_input("Enter Task ID to mark completed: ").strip()
            if not task_id_str.isdigit():
                print("Error: Task ID must be a numeric value.")
                continue
            mark_task_completed(int(task_id_str))

        elif choice == '5':
            task_id_str = custom_input("Enter Task ID to delete: ").strip()
            if not task_id_str.isdigit():
                print("Error: Task ID must be a numeric value.")
                continue
            delete_task(int(task_id_str))
```

```
            elif choice == '6':
                print("Exiting To-Do List Manager. Goodbye!")
                break

            else:
                print("Error: Invalid choice. Please enter a digit from 1 to 6.")

        except Exception as e:
            print(f"Unexpected error occurred: {e}")

if __name__ == '__main__':
    # Sample run simulating user interactions
    sample_inputs = [
        "1", "Read Chapter 4 Database Indexing", "2026-10-26",
        "2",
        "4", "1",
        "3",
        "6"
    ]
    main_loop(simulated_inputs=sample_inputs)
```

- **Output:-**

```
==============================
     To-Do List Manager
==============================
1. Add New Task
2. View Pending Tasks
3. View All Tasks (including completed)
4. Mark Task as Completed
5. Delete Task
6. Exit
==============================
Enter your choice: 1
Enter task description: Read Chapter 4 Database Indexing
Enter due date (YYYY-MM-DD): 2026-10-26
Success: Task #1 added.

==============================
     To-Do List Manager
==============================
1. Add New Task
2. View Pending Tasks
3. View All Tasks (including completed)
```

```
4. Mark Task as Completed
5. Delete Task
6. Exit
================================
Enter your choice: 2

ID   Status     Due Date      Description
-------------------------------------------------------
1    [TODO]     2026-10-26    Read Chapter 4 Database Indexing
-------------------------------------------------------

================================
     To-Do List Manager
================================
1. Add New Task
2. View Pending Tasks
3. View All Tasks (including completed)
4. Mark Task as Completed
5. Delete Task
6. Exit
================================
Enter your choice: 4
Enter Task ID to mark completed: 1
Success: Task #1 marked as completed.

================================
     To-Do List Manager
================================
1. Add New Task
2. View Pending Tasks
3. View All Tasks (including completed)
4. Mark Task as Completed
5. Delete Task
6. Exit
================================
Enter your choice: 3

ID   Status     Due Date      Description
-------------------------------------------------------
1    [DONE]     2026-10-26    Read Chapter 4 Database Indexing
-------------------------------------------------------
```

```
==============================
     To-Do List Manager
==============================
1. Add New Task
2. View Pending Tasks
3. View All Tasks (including completed)
4. Mark Task as Completed
5. Delete Task
6. Exit
==============================
Enter your choice: 6
Exiting To-Do List Manager. Goodbye!
```